

Hooking up the resources

Once we have the resource classes, we can hook the resources in *urls.py* so that it generates API end points for each resource. Let's create *urls.py* in our *books* app, as follows:

```
from django.conf.urls import url, include
from tastypie.api import Api
from books.api import BookResource, PublicationResource
api = Api(api_name='v1')
api.register(PublicationResource())
api.register(BookResource())
urlpatterns = [
    url(r'api/', include(api.urls)),
]
```

Include *books/urls.py* in *my_api/urls.py* as follows:

```
urlpatterns = [
    url(r'^admin/', admin.site.urls),
    url(r'^', include('books.urls')),
]
```

Once we are done with these steps, we are ready to play with Tastypie and fire up the REST API for books and publications.

Before running the server, let's first run the *migrations* command, which will sync models in the DB.

```
./manage.py makemigrations
./manage.py migrate
./manage.py runserver
```

Using REST API URLs

Open another terminal and run a *Curl* command to hit API URLs.

```
curl http://localhost:8000/api/v1/books/?format=json
```

It will return an output as follows:

```
{
  "meta": {
    "limit": 20,
    "next": null,
    "offset": 0,
    "previous": null,
    "total_count": 0,
    "objects": []
  }
}
```

In this output, we have an empty 'objects' list because we don't have any books yet. So let's use the POST method to add book records.

Creating new resources

Before adding a book resource, let's first add one publication resource, which will be attached to the book, as the Book model has the *publication* field as a foreign key to the Publication model.

```
curl -H "Content-Type: application/json" -X POST -d '{"name": "HarperCollins", "isbn": "123-456", "website": "https://harpercollins.co.in/"}' -i http://localhost:8000/api/v1/publications/
```

```
harpercollins.co.in/"}' -i http://localhost
```

The above commands result in the following output:

```
HTTP/1.0 401 Unauthorized
Date: Sun, 09 Oct 2016 13:10:02 GMT
Server: WSGIServer/0.1 Python/2.7.12
X-Frame-Options: SAMEORIGIN
Content-Type: text/html; charset=utf-8
```

Oops! We are getting 401 status because, for reasons linked to safety, Tastypie ships with an *authorisation* class that is set to *ReadOnlyAuthorization*. This makes it safe to expose the API on the Web and to prevent anyone from doing POST/PUT/DELETE. Let's enable this in our resources, as follows:

```
from tastypie.authorization import Authorization
from tastypie.resources import ModelResource
from books.models import Book, Publication
class PublicationResource(ModelResource):
    """Resource for Publication
    """
    class Meta:
        queryset = Publication.objects.all()
        resource_name = 'publications'
        authorization = Authorization()
class BookResource(ModelResource):
    """Resource for Book
    """
    class Meta:
        queryset = Book.objects.all()
        resource_name = 'books'
        authorization = Authorization()
```

Warning

Enabling authorisation is great for testing, but it should only be used during development, and the API should not be exposed to the Web with such resources. I have given a link for authorisation/authentication in the *References* section. Please spend some time in understanding those concepts and learn how to integrate them in Tastypie after reading this article.

So, let's now add a publication resource using POST:

```
curl -H "Content-Type: application/json" -X POST -d '{"name": "HarperCollins", "isbn": "123-456", "website": "https://harpercollins.co.in/"}' -i http://localhost:8000/api/v1/publications/
```

What follows is the output resulting from the above command:

```
HTTP/1.0 201 Created
```